Reviewer Pack — Minimal Order of Monodromy Representations in Hodge Theory a…

Entry 1ff34ad05d3d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 04:33:31 UTC

Minimal Order of Monodromy Representations in Hodge Theory and Communication Complexity Rank

Entry ID: `1ff34ad05d3d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 04:33:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Communication Complexity

Statement:

> The minimal order of monodromy representations of a given curve over the algebraic closure of the rationals is linearly correlated with the communication complexity rank of any circuit computing an equivalent representation, such that the order of the monodromy group |G| = Θ(rank(C)) for circuits C of size n.

Rationale (proposer's reasoning):

> Hodge Theory provides a rich algebraic-geometric framework for studying the behavior of complex functions. Monodromy representations are invariant under certain transformations of curves, which can be related to computational processes such as communication complexity. By examining the minimal order of these representations, we may uncover a new connection between geometric invariants and circuit complexity.

Taxonomy category: `arithmetic_hierarchy` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `70a5510f7db8f6cf`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> A conjecture is supported if, for all generated circuits C with size n ≤ 40, the ratio of the minimal order of monodromy group |G| to the communication complexity rank (rank(C)) falls within a 95% confidence interval of 1, and no seed produces a metric value for either |G| or rank(C) greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Theory" AND "communication complexity" AND "monodromy representations" - "algebraic geometry" AND "communication complexity" AND "circuit rank" - "minimal order monodromy" AND "communication complexity" AND "circuit size"

Top relevant hits considered: - [s2:2502.10590] Optimal Bounds for the Number of Pieces of Near-Circuit Hypersurfaces - [s2:1002.0145] From Sylvester-Gallai Configurations to Rank Bounds: Improved Black-Box Identity Test for Depth-3 Circuits - [s2:10.1145/3548606.3559389] Fast Fully Secure Multi-Party Computation over Any Ring with Two-Thirds Honest Majority

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        # Generate a random circuit of size n
        return [random.randint(1, 3) for _ in range(n)]

    def compute_communication_complexity(circuit):
        # Compute the communication complexity rank of the circuit
        # This is a placeholder function; replace with actual computation
        return len(set(circuit))

    def compute_monodromy_group_order(curve):
        # Compute the minimal order of monodromy representations for the curve
        # This is a placeholder function; replace with actual computation
        return random.randint(1, 20)

    n_max = 40
    instances_tested = 30
    metric_values = []

    for _ in range(instances_tested):
```

```
        circuit = generate_circuit(n_max)
        rank = compute_communication_complexity(circuit)
        order = compute_monodromy_group_order(circuit)

        if rank > 10 or order > 10:
            return {
                "metric_name": "Monodromy Group Order / Communication Complexity Rank",
                "metric_value": None,
                "instances_tested": instances_tested,
                "n_max": n_max,
                "conjecture_holds": False,
                "counterexample": f"Circuit of size {n_max} with rank {rank} and order {order}"
            }

        metric_values.append(order / rank)

    mean_metric = sum(metric_values) / instances_tested
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / instances_tested)

    return {
        "metric_name": "Monodromy Group Order / Communication Complexity Rank",
        "metric_value": mean_metric,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": all(0.95 <= x <= 1.05 for x in metric_values),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(r
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results if r["metri

    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Circuit with rank and order exceeding 10\" firs
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)


```
Rank', 'metric_value': None, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counter
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
TRIAL: {'metric_name': 'Monodromy Group Order / Communication Complexity Rank', 'metric_value': None,
```


8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents a proper evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for a valid assessment of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19047
2	propose	ollama_remote	glm4:latest	0	0	18651
3	propose	ollama_remote	glm4:latest	0	0	17964
4	propose	ollama_remote	glm4:latest	0	0	23003
5	preregistration	ollama_remote	glm4:latest	0	0	12388
6	novelty	ollama_remote	glm4:latest	0	0	8514
7	novelty	ollama_remote	glm4:latest	0	0	11557
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12317
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8698
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13694
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9458
12	judge	ollama_remote	glm4:latest	0	0	11538

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 166829 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/1ff34ad05d3d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1ff34ad05d3d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1ff34ad05d3d.tar.gz` (if generated)